CEBU INSTITUTE OF TECHNOLOGY
UNIVERSITY

IT342-G4
System Integration and Architecture

System Design Document (SDD)

Project Title: QueueEase

Prepared By: Alia, Myron Deandre G.

Version: 2.0

Date: 3/3/2026
Status: Draft

REVISION HISTORY TABLE

Version	Date	Author	Changes Made	Status
1.0	2/18/2026	Alia, Myron Deandre	Initial draft	Draft
2.0	3/3/2026	Alia, Myron Deandre	Added Database Schema , UI Wireframes, Fixed role definitions, Firestore terminology, queue monitoring description, endpoint completeness, and formatting issues	Review

TABLE OF CONTENTS

Contents

EXECUTIVE SUMMARY 4
1.0 INTRODUCTION 5
2.0 FUNCTIONAL REQUIREMENTS SPECIFICATION 5
3.0 NON-FUNCTIONAL REQUIREMENTS 9
4.0 SYSTEM ARCHITECTURE 11
5.0 API CONTRACT & COMMUNICATION 12
6.0 DATABASE DESIGN 17
7.0 UI/UX DESIGN 18
8.0 PLAN 21

EXECUTIVE SUMMARY

1.1 Project Overview

QueueEase is a service-oriented queue management system designed to digitally organize customer queues for service centers such as offices, clinics, and customer service counters. The system allows users to register, authenticate, join queues, and monitor their queue status, while authorized staff and administrators manage queue operations and system data.

QueueEase is implemented using a **Spring Boot backend** that follows a layered architecture, a **React-based web application**, and an **Android mobile application**. The system integrates **Firebase Firestore** as its primary database, incorporates **JWT-based authentication**, **Google OAuth login**, **email notifications**, **file uploads**, and **sandbox payment gateway integration**, in compliance with course requirements.

1.2 Objectives

1. Develop a complete Queue Management that satisfies all required IT342 system features

2. Implement secure authentication using JWT, BCrypt, and Google OAuth login

3. Apply role-based access control for administrators, staff, and regular users

4. Integrate external services including email delivery, file uploads, and payment gateway (sandbox mode)

5. Ensure the system follows a structured backend architecture with proper separation of concerns

6. Deliver a responsive and cross-platform application accessible via web and mobile devices

1.3 Scope

Included Features

- User registration, login, logout, and authenticated session management
- Google OAuth login with internal JWT generation
- Role-based access control (Admin, Staff, User)
- Service center and queue management (full CRUD operations)
- Real-time or near real-time queue updates
- File upload for service-related documents or attachments
- Email notifications for account and system events
- Sandbox payment gateway integration (where applicable)
- External public API integration for additional system data
- Responsive web interface and Android mobile application
- Firebase Firestore database with normalized data structure

Excluded Features

- AI-based decision making
- Production-mode payment processing
- Multi-language support

1.0 INTRODUCTION

1.1 Purpose

This document defines the system requirements, architecture, and functional behavior of the QueueEase application. It serves as a reference for development, testing, and evaluation, ensuring that the system meets all mandatory implementation requirements for the IT342 course and is ready for final defense.

2.0 FUNCTIONAL REQUIREMENTS SPECIFICATION

2.1 Project Overview

Project Name: QueueEase

Domain: Digital Queue Management

Primary Users: Customers, Service Staff, System Administrators

Problem Statement:

Manual queuing systems cause long wait times, inefficient service handling, and poor customer experience**.**

Proposed Solution:

QueueEase digitizes the queuing process by allowing customers to queue remotely and enabling staff to manage queues through a centralized system.

2.2 Core User Journeys

Journey 1: First-Time Customer

1. User registers using email/password or Google OAuth
2. User is authenticated and issued a JWT
3. User views available service centers
4. User joins a queue
5. User monitors queue position

Journey 2: Returning Customer

1. User logs in
2. User joins or cancels a queue
3. User receives system email notifications
4. User completes service and exits queue

Journey 3: Staff Queue Management

1. Staff logs in

2. Staff manages service centers and queues (CRUD)
3. Staff uploads service-related files
4. Staff views payment records and system data

2.3 Feature List (MoSCoW)

MUST HAVE

1. User authentication with JWT and BCrypt
2. Google OAuth login
3. Role-based access control
4. Core queue management module (CRUD)
5. External API integration
6. File upload and retrieval
7. Email sending via SMTP
8. Payment gateway integration (sandbox mode)

SHOULD HAVE

1. Real-time queue updates
2. Queue history tracking
3. Input validation and error handling

COULD HAVE

1. Estimated wait time display
2. Favorite service centers

WON'T HAVE

1. Production payment processing

2. AI-based queue prediction

2.4 Detailed Feature Specifications

Feature: Authentication & Security

- User registration and login
- Google OAuth authentication
- JWT issuance and validation
- BCrypt password hashing
- /me endpoint for authenticated user retrieval
- Protected routes for secured pages

Feature: Role-Based Access Control

- Minimum roles: Admin, Staff, and User
- API-level access restrictions
- UI-level feature restrictions

Feature: Queue Management (Core Business Module)

- Entity: Queue / ServiceCenter
- Full CRUD operations
- Proper request validation
- Linked records in database

Feature: Queue Monitoring

- Real-time queue position tracking
- Firestore listener integration for live updates
- Display of estimated wait time
- Queue status updates (WAITING, CALLED, COMPLETED, CANCELLED)

Feature: External API Integration

- Consumption of a public API
- Display of retrieved data within system UI

Feature: File Upload

- Upload of images or documents
- Server-side storage
- File linked to database records
- Download and viewing support

Feature: Payment Integration

- Real payment provider (test mode only)
- Transaction result storage
- Success and failure handling

Feature: Email Notifications

- Account-related email (e.g., welcome email)
- System notification email (e.g., queue confirmation)

2.5 Acceptance Criteria

AC-1: Secure User Registration

Given a new user

When valid credentials or Google OAuth are used

Then the user account is created and authenticated

AC-2: Queue Participation

Given an authenticated user

When the user joins a queue

Then the queue position is recorded and displayed

AC-3: Role-Based Access Enforcement

Given a non-admin user

When accessing admin functions

Then access is denied

AC-4: Payment Handling

Given a sandbox payment is processed

Then the result is stored and reflected in the system

3.0 NON-FUNCTIONAL REQUIREMENTS

3.1 Performance Requirements

- API responses within 2 seconds for most requests
- Support at least 100 concurrent users
- Efficient database read/write operations

3.2 Security Requirements

- HTTPS enforced
- JWT-based authentication
- BCrypt password hashing
- Protected routes and endpoints
- No exposure of sensitive data via DTOs

3.3 Compatibility Requirements

- Modern web browsers
- Android API level 24 and above
- Responsive layout across screen sizes

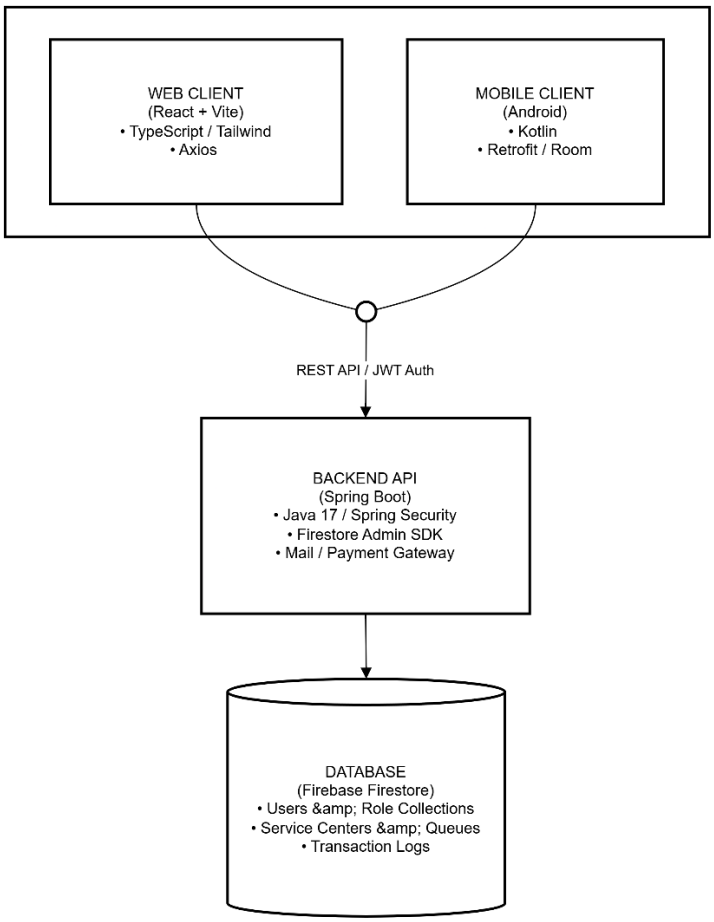
3.4 Usability Requirements

- Simple and intuitive navigation
- Clear feedback for user actions
- Accessibility-aware interface design

4.0 SYSTEM ARCHITECTURE

4.1 Component Diagram

Note: This should be a component diagram



Technology Stack:

- **Backend:** Java 17, Spring Boot 3.x, Spring Security (JWT & Google OAuth 2.0).
- **Database:** Firebase Firestore (NoSQL) for real-time queue management and normalized data structure.
- **Web Frontend:** React 18 using **Vite** for the build pipeline, TypeScript, Tailwind CSS, and Axios.
- **Mobile:** Kotlin and Retrofit for API communication.
- **Build Tools:** Maven (Backend), npm/yarn (Web), Gradle (Android)
- **Deployment:** Railway/Heroku (Backend), Vercel/Netlify (Web), APK (Mobile)
- **External Integrations:** Google OAuth 2.0 (Authentication), SMTP (Email), Firebase Storage (File Uploads), and Sandbox Payment Gateway.

5.0 API CONTRACT & COMMUNICATION

5.1 API Standards

Base URL	https://[server_hostname]:[port]/api/v1
Format	JSON for all requests/responses
Authentication	Bearer token (JWT) in Authorization header

Response Structure	{
	"success": boolean,
	"data": object | null,
	"error": {
	"code": string,
	"message": string,
	"details": object | null
	},
	"timestamp": string
	}

5.2 Endpoint Specifications

Authentication Endpoints

User Registration

Field	**User Registration**
Description	**Creates a new account for a customer or business staff member.**
API URL	**/auth/register**
HTTP Method	**POST**

Field	User Registration
Format	JSON for all requests/responses
Authentication	None (Public Endpoint)
Request Payload	{ "email": <string>, "password": <string>, "firstname": <string>, "lastname": <string> }
Response Structure	{ "success": boolean, "data": { "user": { "email": <string>, "firstname": <string>, "lastname": <string> }, "accessToken": <token>, "refreshToken": <token> }, "error": { "code": string, "message": string, "details": object|null }, "timestamp": string }

User Login

Field	User Login
Description	Authenticates a user and issues security tokens (JWT).
API URL	/auth/login
HTTP Method	POST
Format	JSON for all requests/responses
Authentication	None (Public Endpoint)
Request Payload	{ "email": <string>, "password": <string> }
Response Structure	{ "success": boolean, "data": { "user": { "email": <string>, "firstname": <string>, "lastname": <string>, "role": <string> }, "accessToken": <token>, "refreshToken": <token> }, "error": {

Field	User Login
	"code": string, "message": string, "details": object|null }, "timestamp": string }

Queue & Service Center Management

Field	Join Queue
Description	Allows an authenticated user to join a specific service center queue.
API URL	/queues/join/{centerId}
HTTP Method	**POST**
Format	JSON for all requests/responses
Authentication	Bearer Token (JWT)
Request Payload	{ "notes": <string>, "attachmentUrl": <string> }
Response Structure	{ "success": boolean, "data": { "queueId": <string>, "position": <number>, "estimatedWait": <string> }, "error": { ... }, "timestamp": string }

Register Business (Admin/Staff)

Field	Register Business (Admin/Staff)
Description	Allows an admin or business owner to register their service center.
API URL	/service-centers
HTTP Method	**POST**

Field	Register Business (Admin/Staff)
Format	JSON for all requests/responses
Authentication	Bearer Token (JWT - Admin/Staff Only)
Request Payload	{ "name": <string>, "description": <string>, "location": <string>, "category": <string> }
Response Structure	{ "success": boolean, "data": { "centerId": <string>, "status": "ACTIVE" }, "error": { ... }, "timestamp": string }

System Integration Endpoints

Field	File Upload
Description	Uploads service-related documents to Firebase Storage.
API URL	/files/upload
HTTP Method	POST
Format	multipart/form-data
Authentication	Bearer Token (JWT)
Request Payload	file: <binary_data>
Response Structure	{ "success": boolean, "data": { "fileUrl": <string>, "fileName": <string> }, "error": { ... }, "timestamp": string }

Process Payment (Sandbox)

Field	Process Payment (Sandbox)
Description	Handles sandbox transaction processing for services.
API URL	/payments/process
HTTP Method	POST
Format	JSON for all requests/responses
Authentication	Bearer Token (JWT)
Request Payload	{ "amount": <number>, "currency": "PHP", "paymentMethod": <string> }
Response Structure	{ "success": boolean, "data": { "transactionId": <string>, "status": "COMPLETED" }, "error": { ... }, "timestamp": string }

5.3 Error Handling

HTTP Status Codes

- 200 OK - Successful request
- 201 Created - Resource created
- 400 Bad Request - Invalid input
- 401 Unauthorized - Authentication required/failed
- 403 Forbidden - Insufficient permissions
- 404 Not Found - Resource doesn't exist
- 409 Conflict - Duplicate resource
- 500 Internal Server Error - Server error

Error Code Examples

Example	JSON Structure

Example 1	{
	"success": false,
	
	"data": null,
	
	"error": {
	
	"code": "AUTH-001",
	
	"message": "Invalid credentials",
	
	"details": "Email or password is incorrect"
	},
	"timestamp": "2024-01-28T10:30:00Z"
	}

Example 2	{
	"success": false,
	
	"data": null,
	
	"error": {
	
	"code": "VALID-001",
	
	"message": "Validation failed",
	
	"details": {
	"email": "Email is required",
	
	"password": "Must be at least 8 characters"
	
	}
	
	},

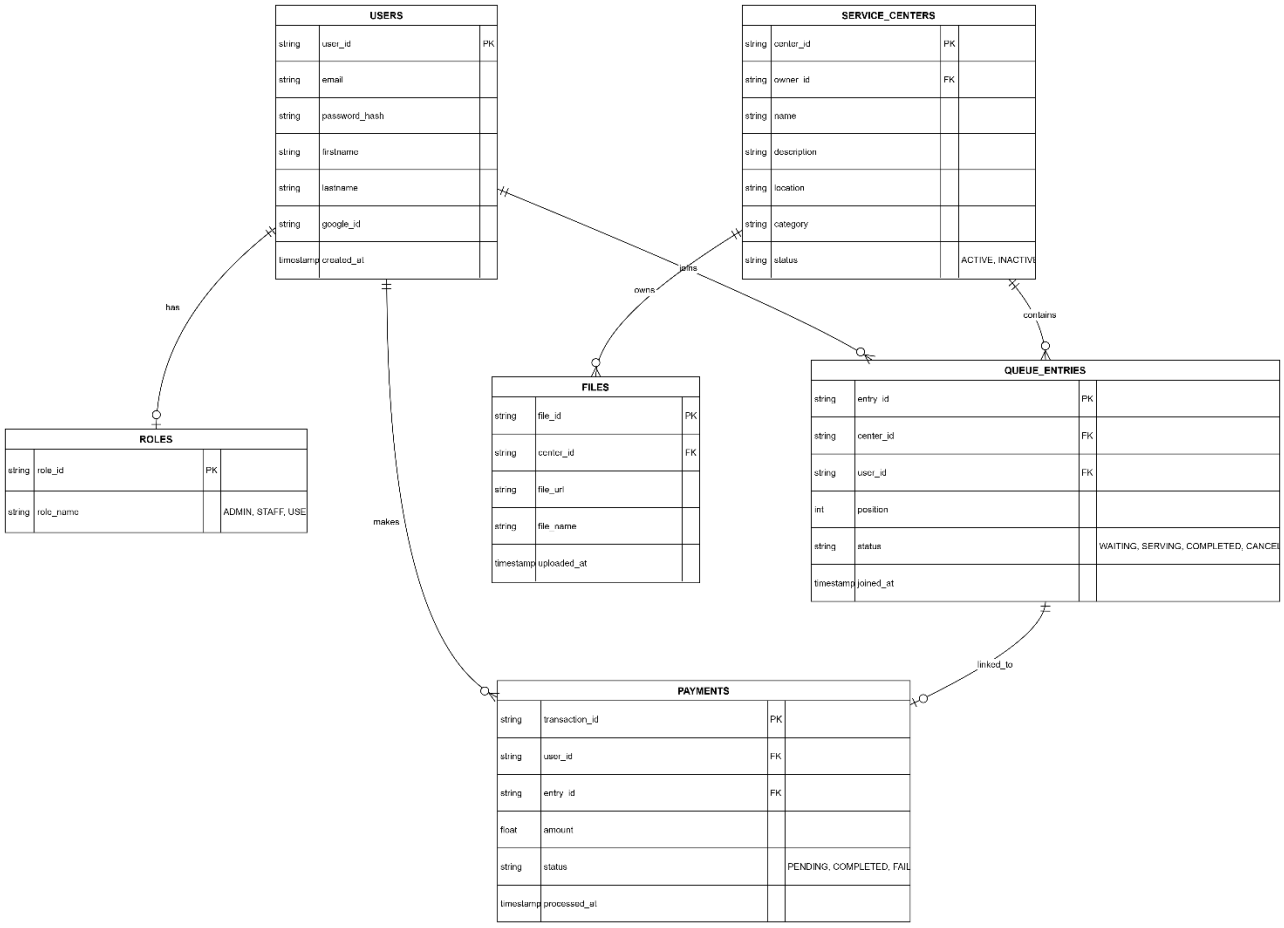
Example	JSON Structure
	"timestamp": "2024-01-28T10:30:00Z" }

Common Error Codes

- AUTH-001: Invalid credentials
- AUTH-002: Token expired
- AUTH-003: Insufficient permissions
- VALID-001: Validation failed
- DB-001: Resource not found
- DB-002: Duplicate entry
- BUSINESS-001: Queue capacity reached
- BUSINESS-002: User already in queue
- BUSINESS-003: Service center inactive
- SYSTEM-001: Internal server error

6.0 DATABASE DESIGN

6.1 Entity Relationship Diagram



Detailed Relationships:

- **One-to-Many: User → Service Centers:** A single user (with Staff/Admin role) can register and manage multiple service centers.
- **One-to-Many: User → Queue Entries:** A customer can join multiple queues (one at a time or historically)
- **One-to-Many: Service Center → Queue Entries:** Each business location manages its own list of customers waiting in line.
- **One-to-Many: Service Center → Files**: Each center can have multiple uploaded documents or images related to its services.

- **One-to-One: Queue Entry → Payment**: Each specific queue transaction is linked to a single sandbox payment record.

Key Collections:

1. **users** - User accounts and authentication
2. **service_centers** - Business/Service center registry and management
3. **queue_entries** - Real-time customer queue tracking
4. **payments** - Transaction records for sandbox payments
5. **files** - Metadata for service-related document uploads
6. **refresh_tokens** - JWT refresh tokens for session management

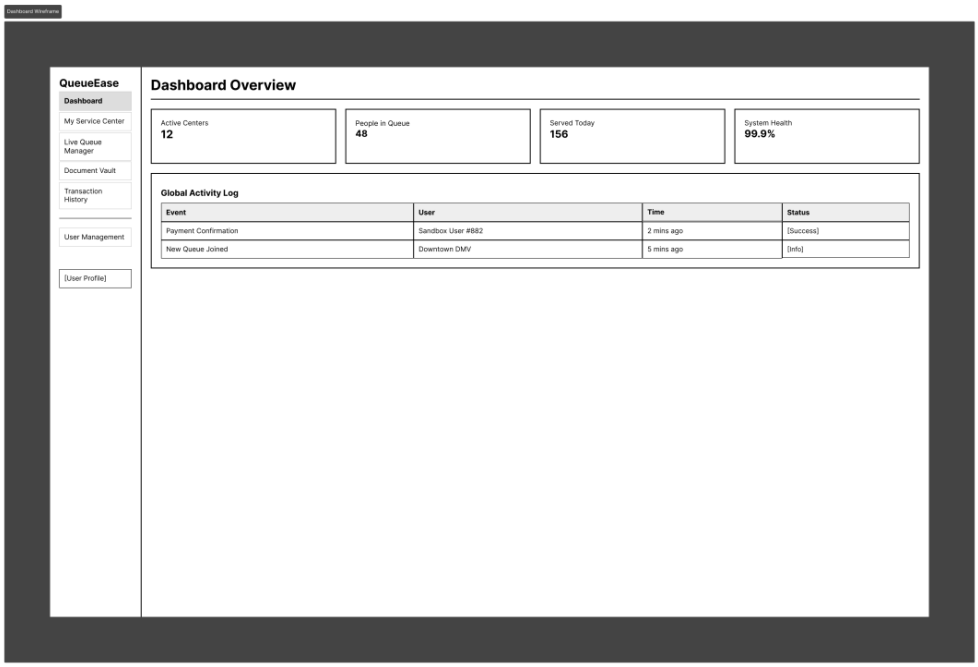
Collection Structure Summary:

- **users**: id, email, password_hash, firstname, lastname, role, created_at
- **service_centers**: id, owner_id, name, description, location, category, status
- **queue_entries**: id, center_id, user_id, position, status, joined_at
- **payments**: id, user_id, entry_id, amount, status, processed_at
- **files**: id, center_id, file_url, file_name, uploaded_at
- **refresh_tokens**: id, user_id, token, expires_at

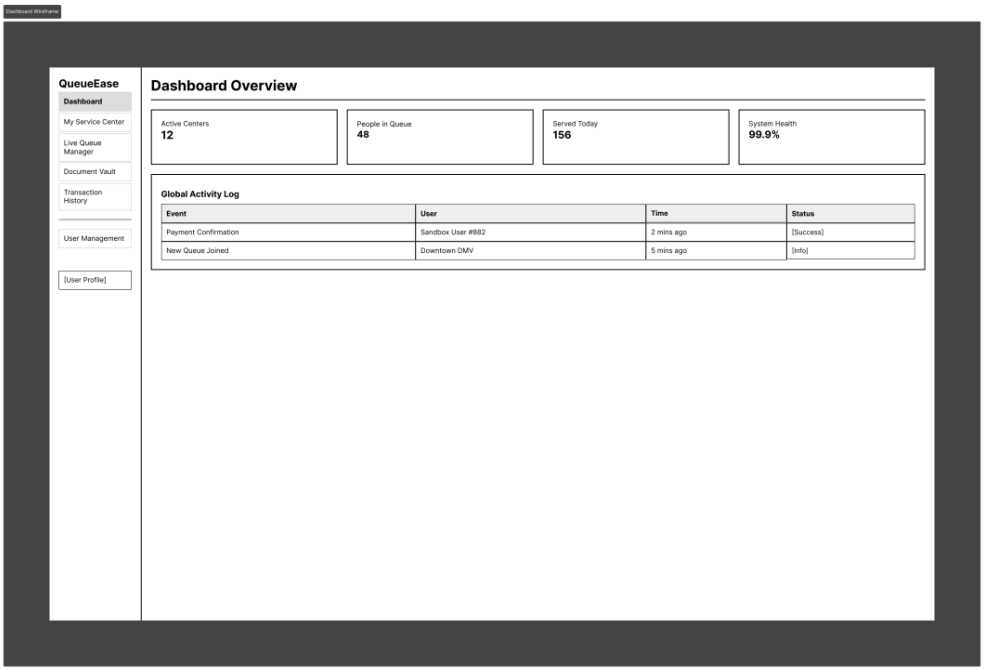
7.0 UI/UX DESIGN

7.1 Web Application Wireframes

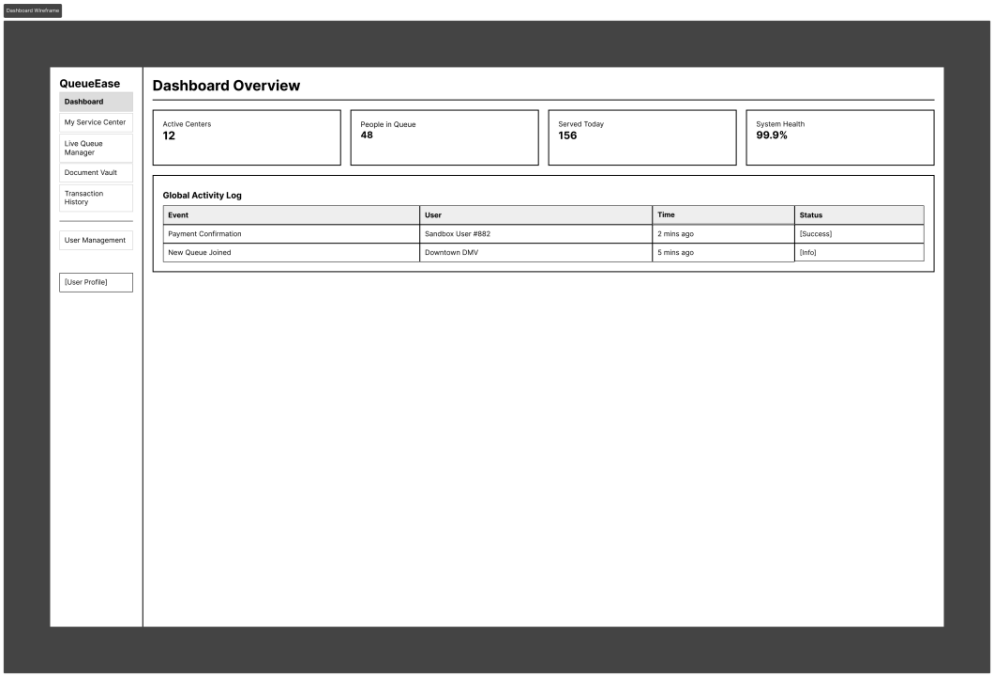
Dashboard Wireframe



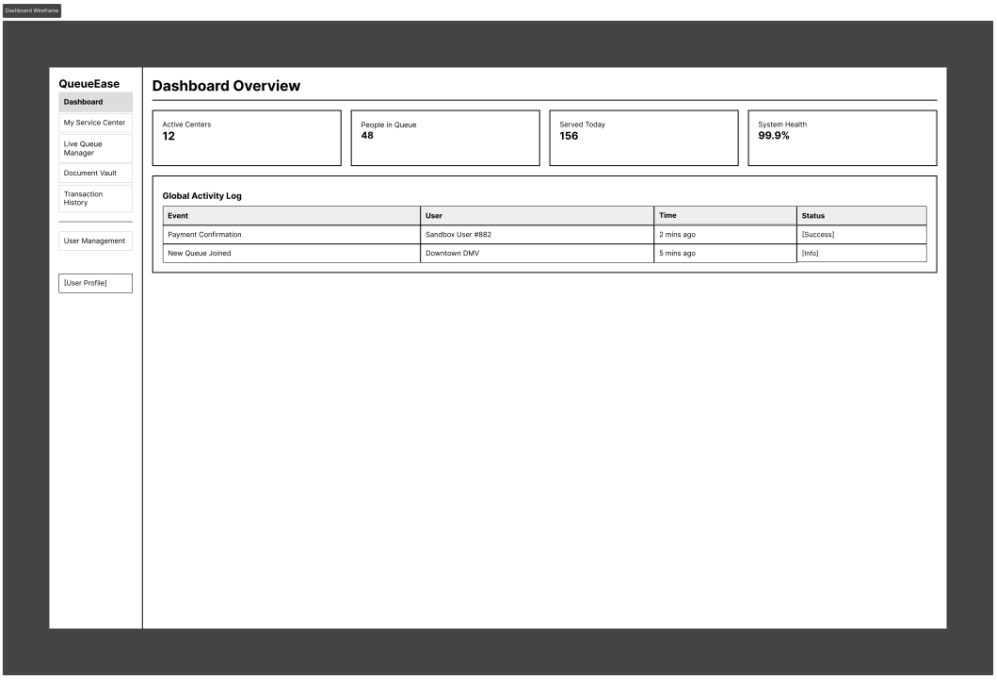
Live Queue Manager Wireframe



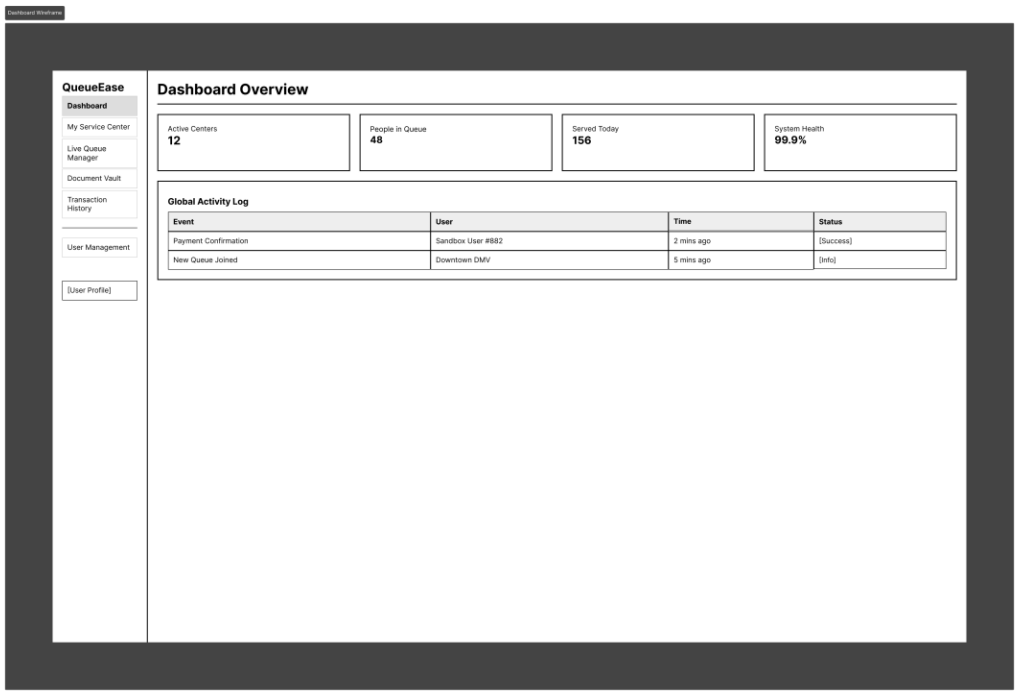
Service Center Management Wireframe



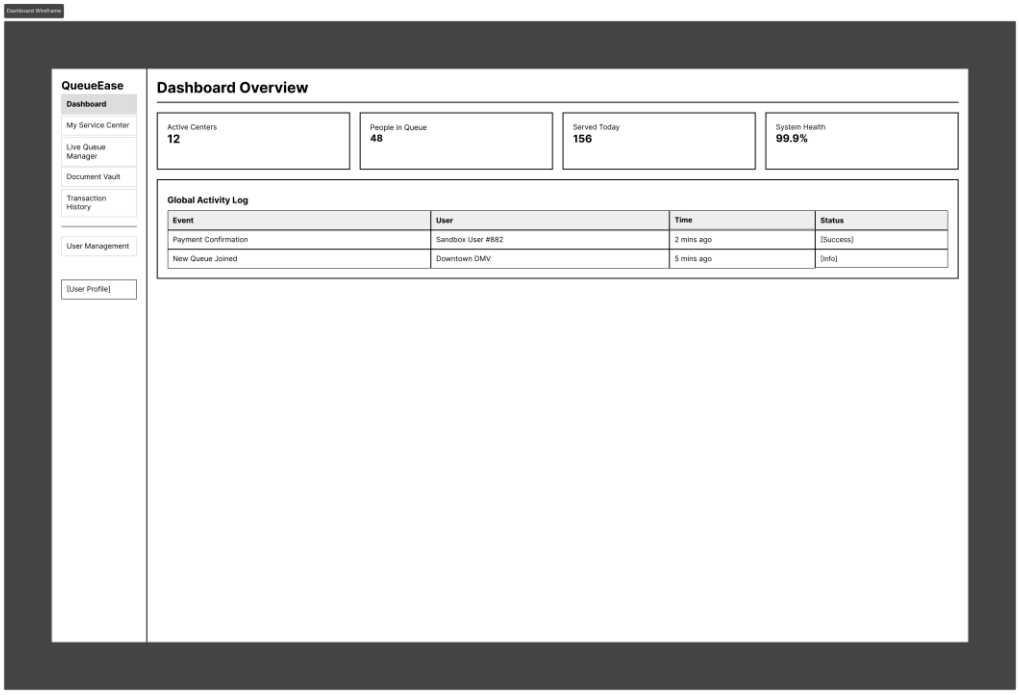
Document Vault Wireframe



Transaction History Wireframe

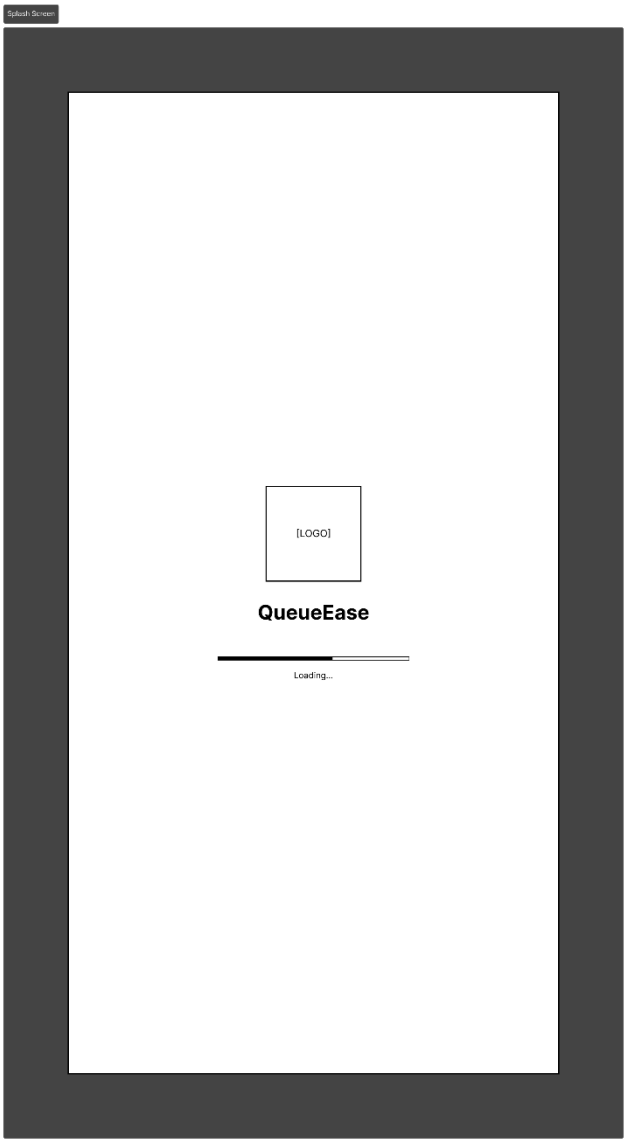


User Management (Admin) Wireframe

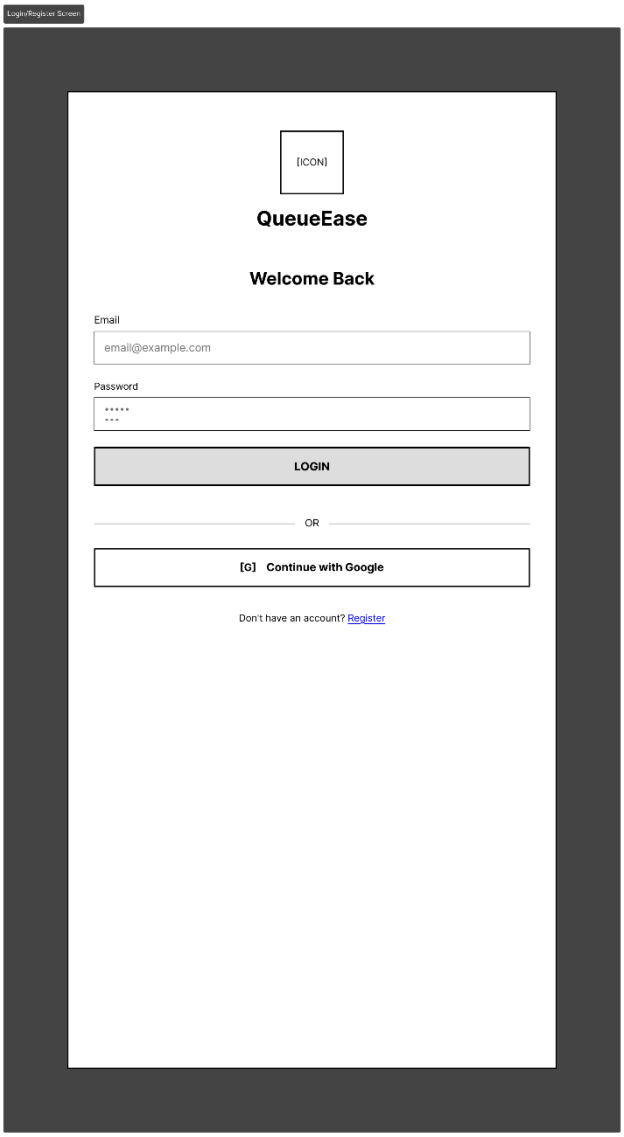


7.2 Mobile Application Wireframes

Splash Screen



Login/Register Screen



Home Screen – Discovery

Home Screen - Discovery

Find Services

Search clinics, offices...

Downtown DMV [Status: Waiting]

#4

Estimated: 15 mins

Nearby Service Centers

[IMG] **City Medical Clinic**
Healthcare
Wait: ~20 mins | [4.5★]

[IMG] **Passport Office**
Government
Wait: ~45 mins | [3.8★]

[IMG] **Bank Branch A**
Finance
Wait: ~10 mins | [4.2★]

[🏠] **Home** [📋] My Queue [👤] Profile

Service Center Detail

Service Center Detail

[←]

[SERVICE IMAGE]

City Medical Clinic

[★ 4.5] Healthcare 0.5 mi

Full-service medical clinic offering general consultations, vaccinations, and health screenings. Walk-ins welcome.

Current Wait

~20 minutes
8 people in queue

Location

[MAP]

123 Main Street, Downtown

Required Documents

Please upload before joining:

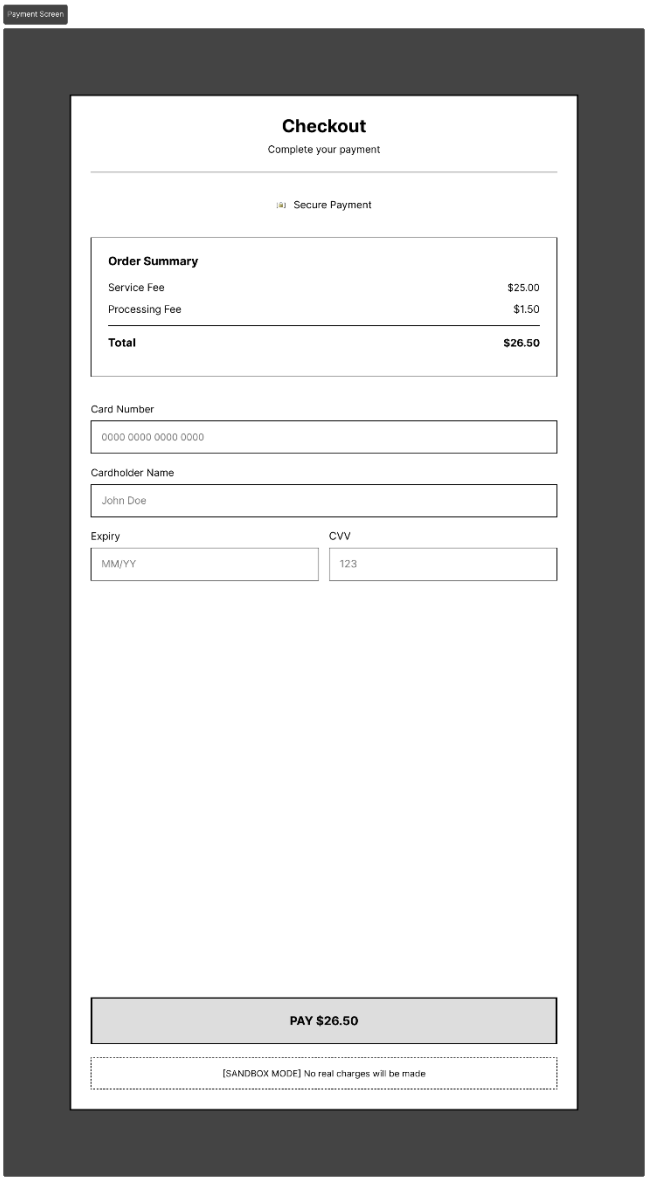
- Valid ID
- Insurance Card (if applicable)

[+] Tap to upload files

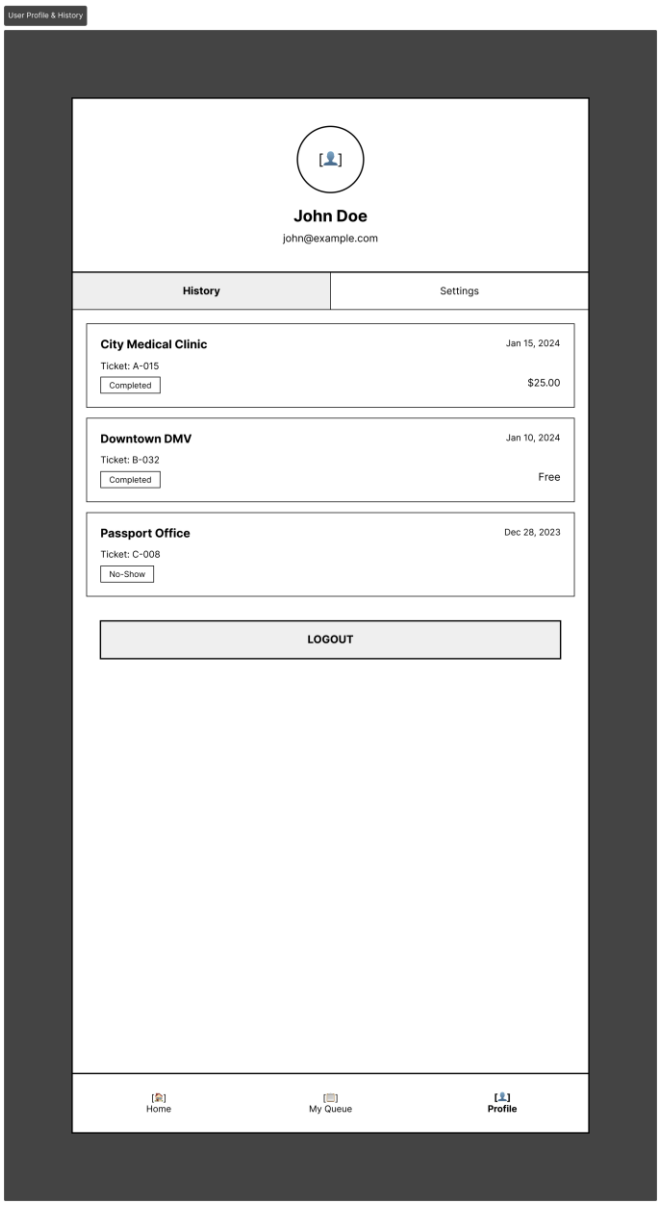
JOIN QUEUE

Queue Monitoring Screen

Payment Screen



User Profile & History



8.0 PLAN

8.1 Project Timeline

Phase 1: Planning & Design (Week 1-2)

Week 1: Requirements & Architecture

Day 1-2: Project setup and documentation

Day 3-4: Complete FRS and NFR

Day 5-7: System architecture design (Component Diagram & Tech Stack)

Week 2: Detailed Design

Day 1-2: API specification

Day 3-4: Database design (Firebase Firestore schema)

Day 5-6: UI/UX wireframes (Vite Web & Kotlin Mobile)

Day 7: Implementation plan finalization

Phase 2: Backend Development (Week 3-4)

Week 3: Foundation

Day 1: Spring Boot setup with Firestore Admin SDK

Day 2: Database configuration and Firestore entities

Day 3: JWT & Google OAuth authentication implementation

Day 4: User management and Role-based access endpoints

Day 5: Service Center CRUD operations

Week 4: Core Features

Day 1: Queue join/cancel functionality

Day 2: Staff queue management (Call Next)

Day 3: Real-time Firestore listeners/updates

Day 4: Error handling and validation

Day 5: API documentation and testing

Phase 3: Web Application (Week 5-6)

Week 5: Frontend Foundation

Day 1: React setup with Vite and TypeScript

Day 2: Admin/Staff authentication pages

Day 3: Service Center dashboard

Day 4: Live Queue Monitor UI

Day 5: File upload implementation (Firebase Storage)

Week 6: Complete Web Features

Day 1: Analytics and transaction logs

Day 2: Email notification triggers (SMTP)

Day 3: Admin control panel

Day 4: Responsive design polish (Tailwind CSS)

Day 5: API integration and testing

Phase 4: Mobile Application (Week 7-8)

Week 7: Android Foundation

Day 1: Android Studio setup (Jetpack Compose)

Day 2: Customer authentication screens

Day 3: Service center browsing and search

Day 4: Queue status and monitoring screens

Day 5: Retrofit API service layer

Week 8: Complete Mobile App

Day 1: Sandbox Payment integration

Day 2: User profile and history

Day 3: UI polish and animations

Day 4: Testing on emulator/device

Day 5: APK generation and documentation

Phase 5: Integration & Deployment (Week 9-10)

Week 9: Integration Testing

Day 1: End-to-end testing (Mobile join -> Web call next)

Day 2: Bug fixes and optimization

Day 3: Security review

Day 4: Performance testing

Day 5: Documentation updates

Week 10: Deployment

Day 1: Backend deployment (Railway/Heroku)

Day 2: Web app deployment (Vercel/Netlify)

Day 3: Mobile APK distribution

Day 4: Final testing

Day 5: Project submission

Milestones:

- **M1 (End Week 2):** All design documents complete
- **M2 (End Week 4):** Backend API fully functional
- **M3 (End Week 6):** Web application complete
- **M4 (End Week 8):** Mobile application complete
- **M5 (End Week 10):** Full system deployed and integrated

Critical Path:

1. Authentication system (Week 3)
2. Queue Management API (Week 3-4)
3. Real-time sync functionality (Week 4)
4. Payment & File integration (Week 6-8)
5. Cross-platform testing (Week 9)

Risk Mitigation:

- Start with simplest working version of each feature
- Test integration points early and often
- Keep backup of working versions
- Focus on core functionality before enhancements